Essaymaster: Keeping a Research Paper Inside the Repository It Describes

Eyal Toledano
eyal@tryhamster.com

August 25, 2026

Abstract

A research paper about an actively developed system is the fastest-rotting document in software: every landed optimization invalidates a number, every redesign invalidates a claim, and unlike ordinary documentation a paper asserts measurements with its author's credibility attached. We describe **essaymaster**, a discipline and an accompanying agent toolkit for *repo-resident* papers. The paper lives in the repository it describes; every number it states flows through a provenance-tagged data pipeline that admits exactly two states (sourced, or loudly flagged `TODO`); and an agent-run maintenance loop detects drift between the paper and the codebase, then folds landed work back into the text.

The discipline was not designed on a whiteboard; it was extracted from five months of practice maintaining the paper of a WebGPU inference engine. We mine that history as a case study: 41 paper-maintenance commits (2026-03 to 2026-08) tracking a codebase that moved by up to 117 commits between paper syncs; a provenance ledger that grew from 258 to 498 rows while its unsourced-row count fell from 16 to 8; review passes whose numbered findings are reconstructible from commit messages; and a one-command, network-free rebuild (data $\rightarrow$ figures $\rightarrow$ PDF) measured at under five seconds, cheap enough to run on every sync. We report what the discipline guarantees structurally, what it merely encourages, and what a single self-reported case study cannot establish. This paper is itself the first end-to-end run of the packaged pipeline; we present that as a demonstration, not as evidence.

1 Introduction

Research papers about living systems occupy an uncomfortable position. They are written like publications, with fixed claims, fixed numbers, and a citation graph, but they describe artifacts that change weekly. The conventional resolutions are all unsatisfying: freeze the paper at submission time and let it drift into fiction; rewrite it wholesale at each milestone and lose the audit trail; or never write it at all, leaving the system's strongest results buried in commit messages and benchmark logs. Software engineering has long known that ordinary documentation rots; Lethbridge et al. found engineers reporting that documentation is usually not updated after code changes [Lethbridge et al., 2003], and Olah and Carter [2017] argue that un-distilled research accumulates a debt of its own. A numbers-bearing systems paper is the worst case of both: it rots like documentation, but its rot is *falsehood* rather than staleness.

Essaymaster is a response built around one structural decision: **the paper is a directory inside the repository it describes**, with the same standing as the code. It is versioned with the code, built by it, reviewed in its pull requests, and updated at the cadence of the work it reports. Around that decision we impose two invariants and one maintenance contract:

- **Invariant 1 (provenance-or-TODO).** Every number in the paper flows from a single curated data file in which each entry carries a provenance string (the file, log, or command it came from, with a date), through a deterministic consolidation script, into the tables and generated figures. A number the authors do not have is entered as the literal value `TODO`, propagates to a flagged row in the audit CSV, and renders in the PDF as a loud red marker. There is no third state, and there is no path by which a number enters the text directly.

- **Invariant 2 (structural honesty).** Unrun measurements are declared unrun in the paper itself, in the abstract and limitations rather than a footnote, and each one gets a committed runbook precise enough that a future session can execute it without re-deriving the design. Negative results are reported with their numbers. Replacing a headline number requires equal-or-better measurement rigor than the number it replaces.
- **The sync contract.** A small state file (`SYNC.json`) records the last repository commit the paper has absorbed and the paths whose changes could affect its claims. Drift is therefore a mechanical quantity, the count of watched commits since the last sync, computable by a shell script cheap enough to run at every agent-session start. Maintenance becomes a classification task over that commit range: changed number, new evidence, narrative shift, retraction trigger, or irrelevant.

The discipline is executed primarily by a coding agent. Essaymaster packages it as an agent skill (a procedure document plus nine reference files encoding the rules above), together with scaffold templates, miner and reviewer subagents, and two drift-detection hooks, for the Claude Code harness [Anthropic, 2025]. We view this packaging through the lens of agent–computer interfaces [Yang et al., 2024]: just as agents resolve issues better when the repository is presented through an interface designed for them [Jimenez et al., 2024], an agent maintains a paper reliably only when the paper's invariants are expressed as an explicit, checkable contract rather than as good intentions.

This paper makes four contributions:

- A **design** for repo-resident papers: the directory contract, the provenance-or-TODO number pipeline, the structural-honesty devices, and the `SYNC.json` drift mechanism (§3).
- An **agent harness** for the design: the skill-and-references encoding, scaffolding templates with a deterministic offline build, and adversarial review with numbered findings (§4).
- A **five-month case study** mined from the git history of the practice the tool was extracted from, the Gerbil WebGPU engine paper [Toledano, 2026b], quantifying maintenance cadence, drift between syncs, provenance-ledger growth, and the TODO burn-down (§5).
- An honest account of the **meta-circular demonstration**: this paper is the packaged pipeline's first end-to-end run, and we delimit exactly what that does and does not show (§6).

2 The Problem: Papers Rot Faster Than Documentation

Three properties make a systems paper harder to keep true than ordinary documentation. First, *claims are quantitative and perishable*: a README that says "fast" survives an optimization campaign; a paper that says "145 tokens/s" does not. In the case study of §5, the flagship decode-throughput figure was refreshed repeatedly across the LaTeX paper's seven-week lifetime, from 145 tokens/s in its first committed abstract to a 347.3 tokens/s kept peak in its last refresh (2.4×). Each refresh was a statement the old text had silently stopped being true. Second, *drift is bursty*: repositories move in campaigns. Between two adjacent paper-maintenance commits in our corpus, up to 117 non-paper commits landed in a 16-day window. A paper synchronized weekly by habit would have been badly wrong mid-window and needlessly touched in quiet weeks; drift needs to be measured, not scheduled. Third, *the failure mode is silent*: nothing breaks when a paper claim goes stale. Code has tests; papers have only their authors' memory. The essaymaster position is that this third property is the one worth engineering away. Staleness must be made *visible* (a drift count printed at session start, and a nudge in the commit output at the moment landed work touches watched paths) and falsehood made *structurally difficult* (no unsourced number can enter the text unflagged).

3 Design: The Paper as a Repository Contract

3.1 The directory contract

A paper under essaymaster is a self-contained directory (`paper/`, or `papers/`⟨`slug`⟩`/` once a repository hosts several) holding the LaTeX source, a verified bibliography, the curated data file, two zero-dependency generator scripts (consolidation and figures), a one-command build, and the sync state. The contract is that the directory builds from a bare checkout with no GPU and no network. Anyone can regenerate every table, figure, and page from the committed data: continuous integration, a teammate, or an agent deciding whether its own edit broke something.

3.2 Invariant 1: the number pipeline

All measured values live in one curated JSON file in which every block carries a provenance string. A consolidation script flattens it into an audit CSV (`key, value, is_todo, provenance`; one greppable row per number) and into the per-figure data series; a figure script renders those series to SVG with no external chart library. The LaTeX source cites numbers that exist in the ledger and includes figures that were generated from it; hand-transcribing a number into the text is a protocol violation that review is instructed to hunt for. The pipeline's value is less automation than *auditability*: "where does this number come from?" is answered by one grep, for every number, forever.

3.3 Invariant 2: structural honesty

Honesty devices that depend on author diligence decay under deadline pressure, so the discipline makes them structural. The `TODO` state is first-class: it survives consolidation, is counted by the build, and renders in red in the PDF, so an unsourced number is never invisible. Pending measurements get a committed *ablation runbook* stating why each matters, the exact protocol, and the instruction that no paper number may be filled from the runbook until the run has actually executed. A *reviewer-readiness checklist* maps every remaining gap to its flagged rows, making "submittable" a checkable predicate rather than a feeling. And replacement of a headline number demands equal-or-better rigor (same or more runs, same protocol), so numbers ratchet toward reliability rather than toward recency.

3.4 The sync contract

`SYNC.json` records `lastSyncedCommit` and `watchedPaths`. Drift is `git log lastSyncedCommit..HEAD -- watchedPaths`, a number a hook can print in one line at session start. A companion git post-commit hook closes the loop from the other side: when a landed commit touches watched paths without touching the paper, the committer (agent or human) is nudged in the commit output itself, so the "does the paper need this?" question is raised at the moment of landing rather than at the next session. A sync is then a bounded editorial procedure: classify each watched commit (changed number, new evidence for an existing claim, narrative shift, retraction trigger, or irrelevant); apply changes *through the data pipeline first*, then the prose; rebuild; advance the sync pointer; commit per theme with a `paper:` prefix so the maintenance history remains legible. Retraction triggers, meaning landed work that falsifies a paper statement, outrank everything else, on the principle that a stale overclaim is worse than any gap.

4 The Agent Harness

The discipline is executed by a coding agent, and the packaging reflects a specific belief: for editorial work, the right agent interface is *prose contracts plus mechanical checks*, not end-to-end automation. The skill's procedure document routes among seven modes (mine, mine-sessions, init, sync, build, review, publish); nine reference files carry the rules of §3 in the imperative, checklist-bearing form agents follow reliably. The mechanical parts live in a single zero-dependency CLI bundled with the skill (drift counting, scaffolding, migration, sync-pointer transitions, bundle assembly, and a `check` verb that lints the invariants: provenance coverage, citation keys, figure files, TODO surfacing, staleness), so the skill routes them through one always-present tool with no fallback branch, and `check` doubles as a continuous-integration gate: the paper gets tests. An agent should never be trusted to *compute* what a script can compute. The judgment parts remain agent work bounded by the invariants: classifying a landed commit, deciding whether a feature is a contribution, wording a qualifier. The per-paper build stays vendored in the paper directory, deliberately: a paper must build from a bare checkout with or without the CLI.

The subagents divide by corpus. The *miner* performs read-only archaeology over the repository: given a slice, it extracts candidate paper material (optimization campaigns with recorded numbers, non-obvious theses in design docs, negative results), each find paired with a web-verified novelty check. The *session-miner* sweeps the agent's own session transcripts for the material git structurally cannot show: measured dead ends that never reached a commit, multi-step diagnosis arcs, and recurring cross-session patterns; its finds are treated as lab-notebook leads that must be re-run or explicitly flagged before entering a ledger, and only session-exclusive material survives the dedupe against git. The *reviewer* is adversarial and edit-free: it returns numbered findings across four lenses (evidence, drift, citations, honesty), which are then fixed in commits referencing the finding ids, so the review trail is reconstructible from history alone (§5 verifies this actually happens in practice). Citation hygiene is its own reference file with one absolute rule: every citation is verified by web search in the

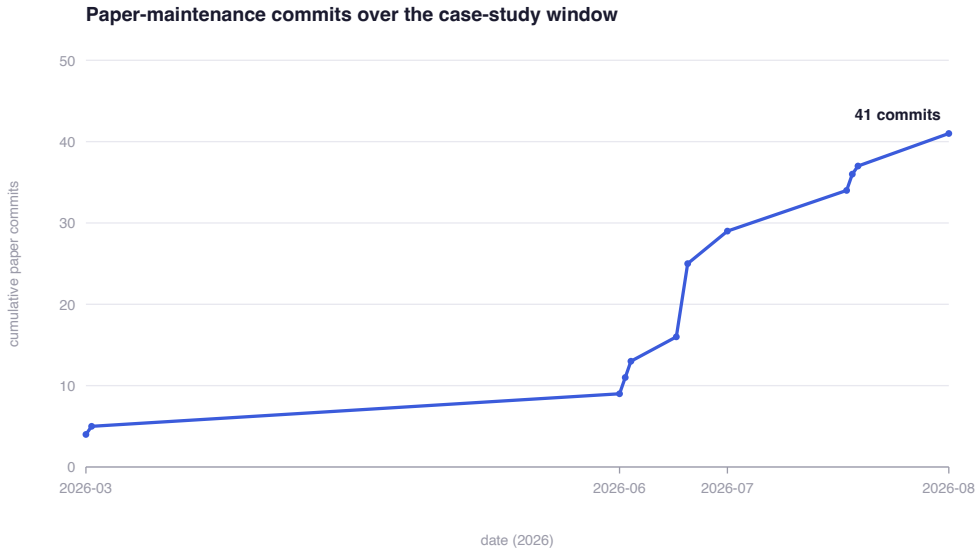

Figure 1: Cumulative paper-maintenance commits in the case-study repository (41 commits, 2026-03-11 to 2026-08-10, current-branch corpus). The staircase shape is the practice's signature: maintenance arrives in bursts on 12 distinct days, up to nine commits in a day, rather than on a schedule. The burst timing is consistent with campaign-driven maintenance (§2); the June cluster also includes the initial planning and drafting work.

session that adds it, and systems without papers are cited as repositories, never as invented arXiv ids.

Init deliberately front-loads planning. Before any LaTeX exists, the agent commits a plan document containing the verified related-work survey, the genre choice, and a *claims-to-required-evidence table* whose rows are the only claims the paper is permitted to make. The paper is then written evidence-first, evaluation skeleton before introduction, so that an unwritable evaluation section is discovered on day one rather than at submission. Finally, the discipline separates *writing* from *publishing*: mined candidates carry a disclosure rating (public, needs-clearance, internal-only), internal-only material never enters a publishable paper without the owner's explicit decision, and an internal whitepaper in a private repository is a first-class target; publication is a distinct, gated step.

5 Case Study: Five Months of the Gerbil Paper

The discipline was extracted from, not imposed on, the paper of the Gerbil WebGPU inference engine [Toledano, 2026b]: a ∼17-page arXiv-style systems preprint that lives in that repository's `paper/` directory. We mined the practice retrospectively on 2026-08-25 from the repository's git history using read-only commands, each recorded alongside its number in this paper's own data ledger. Two scope caveats apply throughout. First, the corpus of "paper-maintenance commits" is every commit touching the paper directory, its engineering-reference precursor document, or the two planning documents, on the current branch: 41 commits from 2026-03-11 to 2026-08-10. The earliest four (March) are engine feature commits that updated the engineering reference as a side effect; the LaTeX paper proper begins 2026-06-23. Second, this is a single practice, mined by the people who ran it; §8 treats the resulting biases.

5.1 Cadence: syncs follow campaigns

Figure 1 shows the maintenance cadence. The paper was touched on just 12 distinct days across five months, in bursts of up to nine commits per day. This is consistent with the design position (§2) that paper maintenance should track campaign boundaries, not calendars. Table 1 quantifies the drift those bursts absorbed: between adjacent paper commits, the repository moved by anywhere from zero to 117 non-paper commits. The measure is an upper bound, since a commit touching both code and paper counts as non-paper, but the shape is the point: drift arrives in avalanches, and a mechanism that *measures* it (the `SYNC.json` count) dominates one that assumes a rate.

Sync commit (date)	Previous sync	Commits between	. . . non-paper
2026-08-10	2026-07-25	120	117
2026-07-25	2026-07-24	14	14
2026-07-24	2026-07-23	30	29
2026-07-23	2026-07-02	38	33
2026-07-02	2026-07-02	15	11

Table 1: Repository movement between selected adjacent paper-maintenance commits (the five largest gaps among the fifteen most recent pairs). "Non-paper" counts commits whose diff touches any watched non-paper path and is an upper bound on drift, since mixed commits count as non-paper.

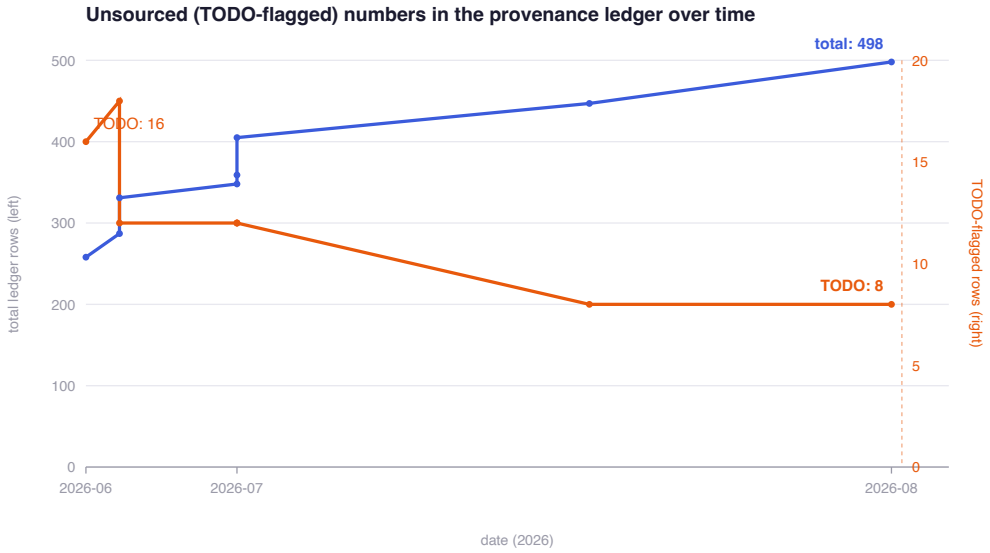

Figure 2: The provenance ledger over its eight committed versions: total rows (blue, left axis) versus the `TODO`-flagged subset (orange, right axis). Rows were added at roughly 150 per month over the ledger's seven-week lifetime while the unsourced subset halved; the transient rise to 18 is a sweep that added new known-unknowns before closing them.

5.2 The provenance ledger: growth with a shrinking gap list

Figure 2 tracks the audit CSV across its eight committed versions. The ledger nearly doubled, from 258 to 498 total rows, while the count of `TODO`-flagged rows among them fell from 16 to 8 (peaking at 18 as an honest sweep added new known-unknowns before closing them); fully sourced rows thus grew from 242 to 490. The residual 8 are not debt hidden in the text: each maps to a declared pending measurement in the repository's committed runbook, and four render as visible red markers in the shipped PDF. This is the intended equilibrium of Invariant 2. The gap list shrinks, but more importantly it is *enumerable* at every point in history by a single grep over a committed file.

5.3 Review passes leave a reconstructible trail

Four commits in the corpus carry review evidence, three of them referencing numbered findings in their subjects, e.g. *"qualify causal claim in abstract+contributions (review #E1/E3/E4)"* and a reframing commit closing findings #E1–E7 and #D1–D2. The finding taxonomy observed in the wild used evidence (#E), drift (#D), and abstract-level (#A) lenses; the citation (#C) and honesty (#H) lenses the packaged reviewer now carries never appear in the historical log. The packaged tool extends the observed practice there, and we flag that as codification rather than extraction. The substantive point survives the caveat: the practice's strongest single honesty event, weakening a causal claim in its own abstract, is a one-line commit anyone can find.

5.4 The build is cheap enough to be unskippable

The case-study paper rebuilds in 4.7 seconds of wall clock on the authoring machine (single run, warm engine cache; indicative, not a benchmark). That rebuild covers a 498-row consolidation, corpus cleaning that logs what it *dropped* (29 adversarial rows, an instance of the no-silent-caps rule), nine SVG figures, vector conversion, and a 17-page PDF via tectonic [The Tectonic Project, 2019]. The number matters only relative to a threshold: rebuild cost this low removes the last excuse for committing an unbuilt paper, so "the PDF in the repo is the current claims" stays true. Snapshot totals at mining time: 498 ledger rows (8 flagged unsourced), 51 verified bibliography entries, 1,365 lines of LaTeX, 9 generated figures, 17 pages.

6 The Meta-Circular Demonstration

This paper was produced as the packaged pipeline's first end-to-end run: a miner subagent extracted every case-study number above from the Gerbil history; seven external citations were web-verified in the authoring session; the paper directory was scaffolded from the shipped templates; the two figures were generated by the zero-dependency pipeline from the provenance ledger (118 rows, 0 flagged); and the PDF builds offline by the standard one-command path. We state plainly what this shows and does not. It demonstrates the pipeline is *executable* end-to-end by an agent, including its verification steps. It is not evidence the discipline improves outcomes, both because a demonstration chosen by the tool's authors proves availability rather than benefit, and because the case study it reports predates the packaging. The packaged tool's own five-month history does not exist yet; when it does, it will be minable by exactly the procedure of §5.

7 Related Work

Documents that live with code. Literate programming made the program and its explanation one artifact [Knuth, 1984]; computational notebooks made the analysis document executable [Kluyver et al., 2016]. Essaymaster inverts the first (the *paper* moves in with the code, keeping its publication genre) and complements the second (the executable part is the pipeline *around* a static PDF, because arXiv-genre papers, not notebooks, are the target artifact). The docs-as-code practice community applies versioning and CI to documentation broadly; essaymaster is that stance specialized to the document class with the strictest truth obligations. Olah and Carter [2017] name the debt; Lethbridge et al. [2003] measured the rot for ordinary documentation two decades ago. **Agents in repositories.** SWE-bench established repository-level issue resolution as an agent task [Jimenez et al., 2024], and SWE-agent showed the interface presented to the agent dominates outcomes [Yang et al., 2024]; essaymaster's skill-plus-invariants packaging is an agent–computer interface for editorial maintenance rather than code change. **Agent-written papers.** The AI Scientist automates the full idea-to-manuscript loop [Lu et al., 2024]. Essaymaster occupies the opposite corner deliberately: papers whose claims are owned by humans and grounded in a repository's recorded measurements, where the agent's autonomy is spent on *maintenance and verification*, the part humans demonstrably neglect, rather than on generation. We found no published treatment of this maintenance problem (keeping a long-lived paper continuously consistent with a moving codebase under machine-checkable provenance) and state that gap as this paper's niche, with the survey-breadth caveat of §8.

8 Limitations

The evaluation is a single case study (n=1), of a practice run by this paper's author, mined and written up with the tool being described: three concentric conflicts we mitigate with recorded extraction commands and stated caveats, not eliminate. The case study validates the *discipline* (it was really followed, for five months, at low cost); it cannot validate the *packaged tool*, which post-dates it, nor compare against ad-hoc maintenance by other teams. The controlled study essaymaster's own methodology would demand is future work, and Claim 6 of our plan's claims table is accordingly marked not-claimable. The drift measure is an upper bound; the build timing is a single warm-cache run; the review-lens taxonomy partially post-dates the evidence for it; and the related-work search, while verified entry-by-entry, was breadth-limited. Adjacent literatures (scientific workflow provenance, artifact evaluation, living reviews) may hold closer neighbors than we found.

9 Conclusion

A paper that lives with its codebase can stay true the way code stays working: by making its freshness measurable, its numbers auditable, and its gaps loud. Five months of one practice suggest the cost is modest (twelve days of touches, a five-second rebuild), and the packaged form makes the discipline transferable to any repository with an agent attached. Whether it transfers *well* is now an empirical question the tool is instrumented to answer about itself.

References

Anthropic. Claude code. https://claude.com/claude-code, 2025. product documentation; the agent harness essaymaster targets.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, et al. Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas (ELPUB)*, pages 87–90, 2016.

Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

George Kour. arxiv-style: A LaTeX style and template for paper preprints. https://github.com/kourgeorge/arxiv-style, 2018. no paper; MIT-licensed NeurIPS-2018-derived preprint class.

Timothy C. Lethbridge, Janice Singer, and Andrew Forward. How software engineers use documentation: The state of the practice. *IEEE Software*, 20(6):35–39, 2003.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Chris Olah and Shan Carter. Research debt. *Distill*, 2(3), 2017. DOI 10.23915/distill.00005.

The Tectonic Project. Tectonic: a modernized, complete, self-contained TeX/LaTeX engine. https://github.com/tectonic-typesetting/tectonic, 2019. no paper.

Eyal Toledano. essaymaster. https://github.com/eyaltoledano/essaymaster, 2026a. the artifact described in this paper.

Eyal Toledano. Gerbil: a pure-WGSL WebGPU engine for on-device language and speech models. https://github.com/tryhamster/gerbil, 2026b. repository; companion preprint in `paper/` of the repository.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. NeurIPS 2024.

A Artifact Appendix

Essaymaster (skill, references, commands, agents, hook, scripts, templates, and this paper's full source and data) is the repository this paper resides in [Toledano, 2026a]; the repository is private at the time of writing, with public release planned alongside any public posting of this paper. Rebuild: `cd paper && ./build.sh` (requires Node; the PDF step requires tectonic or a TeX Live engine; figures use rsvg-convert when present). Every case-study number is a row in `results/measurements.csv` whose provenance field records the exact extraction command; the mining was performed against `github.com/tryhamster/gerbil` at branch `docs/paper-two-column` on 2026-08-25. The paper class is the MIT-licensed `arxiv.sty` derivative shipped with the templates [Kour, 2018].